

# Fluid project report

This report aims at presenting the results obtained while working on the fluid simulation project. We present our results, along with the generated images, rendering times and explanations on why the code works. This project contains Voronoi parallel linear enumeration with Sutherland-Hodgman polygon clipping, power diagrams (Laguerre cells), semi-discrete optimal transport with an air cell, the particle update used for the fluid, and frame rendering. All frames use  $800 \times 800$  pixels.

## Voronoi diagram

We start from a set of sites  $P_i$  in the unit square. The Voronoi cell of  $P_i$  is the set of points that are closer to  $P_i$  than to every other site. The implementation follows Voronoi parallel linear enumeration: for each site, the code starts from the whole square, and clips the current polygon against the bisector of  $P_i$  and every other point  $P_j$ :

$$\text{Cell}(P_i) = \left\{ x \in [0, 1]^2 : \|x - P_i\|^2 \leq \|x - P_j\|^2 \quad \forall j \neq i \right\}$$

This is equivalent to keeping one side of an affine line. The clipping step is the Sutherland-Hodgman polygon clipping algorithm, the implementation loops over the edges of the current polygon, and when an edge crosses the bisector, the intersection point is inserted before continuing the clipping step. This gives one polygon per particle.

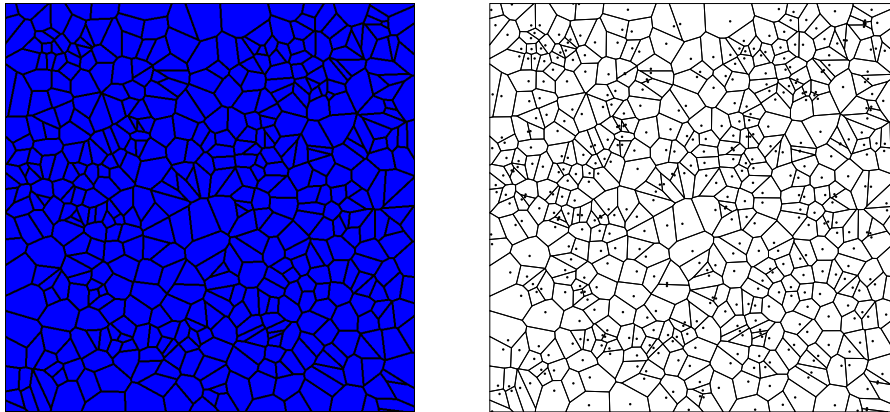


Figure 1: Voronoi diagram with 500 points

## Polygon area and centroid

The area of a polygon is computed by looping over its consecutive vertices, and summing the usual cross-product terms:

$$A = \frac{1}{2} \left| \sum_i x_i y_{i+1} - x_{i+1} y_i \right|$$

Similarly, we compute the centroids as

$$C_x = \frac{1}{6A} \sum_i (x_i + x_{i+1})(x_i y_{i+1} - x_{i+1} y_i), \quad C_y = \frac{1}{6A} \sum_i (y_i + y_{i+1})(x_i y_{i+1} - x_{i+1} y_i)$$

These two formulas are used several times: the area is the mass constraint for optimal transport, and the centroid is used by the fluid force.

## Integral of squared distance

For the optimal transport objective, we need the integral of  $\|x - P_i\|^2$  over each cell. We decompose each cell into triangles, from its first vertex, and for each triangle  $(c_1, c_2, c_3)$  with  $d_k = c_k - P_i$ , the integral is

$$\int_T \|x - P_i\|^2 dx = \frac{|T|}{12} (d_1 \cdot d_1 + d_1 \cdot d_2 + d_1 \cdot d_3 + d_2 \cdot d_2 + d_2 \cdot d_3 + d_3 \cdot d_3)$$

where  $|T| = |(c_2^x - c_1^x)(c_3^y - c_1^y) - (c_2^y - c_1^y)(c_3^x - c_1^x)|$  is twice the triangle area. This avoids numerical sampling, so the same input gives the same output every time.

## Semi-discrete optimal transport

We want to distribute mass uniformly across cells so that each cell has target area  $\lambda = \frac{1}{n}$ . We achieve this by introducing Laguerre weights  $w_i$ , which shift the bisectors. The Laguerre cell of  $P_i$  replaces the Euclidean distance condition by

$$\text{Cell}_w(P_i) = \left\{ x : \|x - P_i\|^2 - w_i \leq \|x - P_j\|^2 - w_j \quad \forall j \neq i \right\}$$

For a prescribed mass  $\lambda = \frac{1}{n}$ , the gradient used by L-BFGS is

$$g_i = \text{area}(\text{Cell}_w(P_i)) - \lambda$$

which vanishes once every cell has the target area. Thus convergence means that the cell areas match the target areas.

After the optimizer converges, we recompute the Voronoi structure with the final weights.

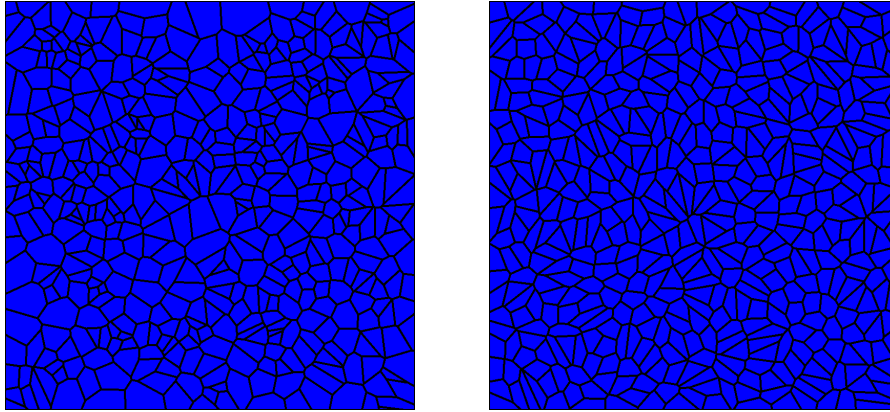


Figure 2: Before and after optimal transport: the weighted cells have equal target area

## Partial optimal transport

For the fluid simulation, we do not want the whole square to be filled with water, so we aim at 25% of the surface instead, so each particle has target area  $\frac{V_{\text{fluid}}}{n}$ .

Partial optimal transport is implemented by adding one extra variable, the air weight  $w_{\text{air}}$ , that controls the boundary between fluid and air. After clipping a cell by the weighted bisectors, we further clip it by a disk centered at the particle of radius  $r_i = \sqrt{w_i - w_{\text{air}}}$ . If  $w_i - w_{\text{air}}$  is non-positive, the cell disappears entirely.

The disk is approximated by a 64-sided polygon, so the existing polygon clipping algorithm can still be used. The additional gradient component is

$$g_{\text{air}} = V_{\text{fluid}} - \sum_i \text{area}(\text{Cell}_w(P_i))$$

which enforces the total amount of water.

## Fluid simulation

At each time step, we solve the partial optimal transport problem using the current particle positions. The force applied to a particle is

$$F_i = m_i g + \frac{1}{\varepsilon^2} (c_i - P_i)$$

where  $c_i$  is the centroid of the transported cell. This term acts like a spring that moves particles toward the centers of their transported volumes, while gravity moves the fluid downward.

Then we update the velocities from the force, and advance the positions with these new velocities:

$$v_i \leftarrow v_i + \frac{dt}{m_i} F_i, \quad P_i \leftarrow P_i + dt \cdot v_i$$

Finally, we reflect particles at the boundary of the unit square. If a coordinate becomes negative, it is mirrored and the corresponding velocity component is reversed. The same rule applies when a coordinate becomes greater than one.

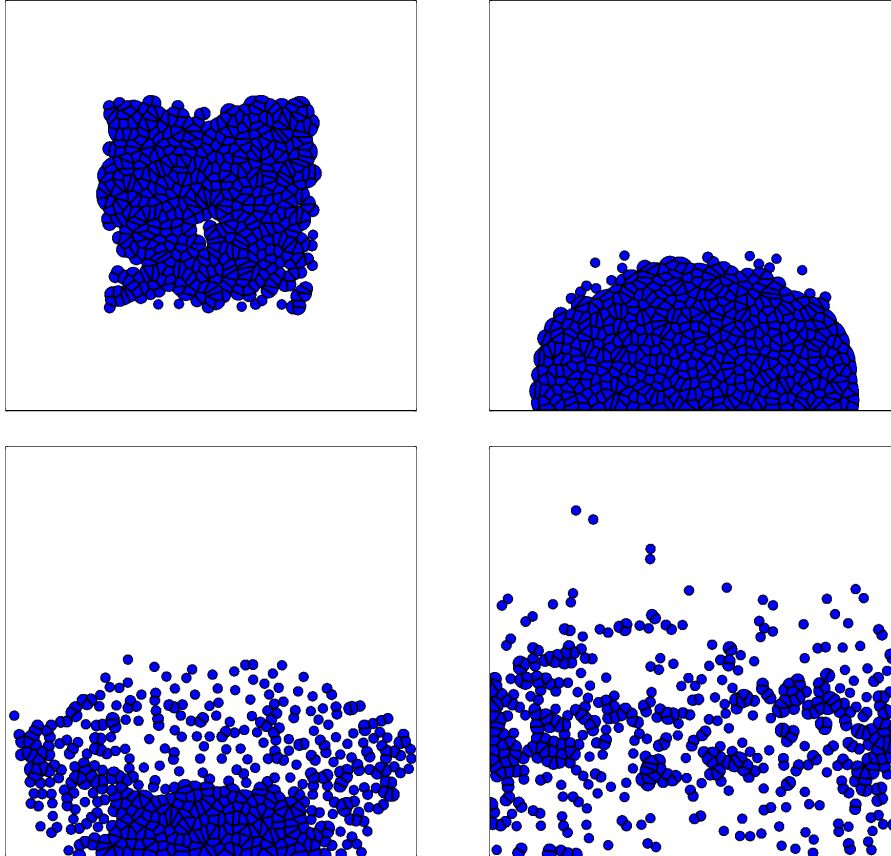


Figure 3: Render of frames 0, 155, 200 and 410 (500 particles)

## Timings

We timed the render times for an increasing number of particles. Each benchmark renders 1000 frames, is run 3 times, and the table reports the average. The timings grow quickly with the number

of particles, which is expected because each cell is clipped against all other particles at every optimization evaluation.

| Particles | 1000 steps | Per step  |
|-----------|------------|-----------|
| 100       | 20.304s    | 0.020304s |
| 250       | 97.869s    | 0.097869s |
| 500       | 481.839s   | 0.481839s |
| 1000      | 2186.928s  | 2.186928s |

## Course feedback

I appreciated the hands-on approach of the course, as it makes the link between the theoretical concepts of the lecture and the actual implementation. The professor and TA were clearly passionate about the subject, which made the lectures enjoyable and the labs engaging.

My main criticism is that the lectures sometimes feel too implementation-heavy, and it feels like we could spend more time understanding a specific concept rather than trying to fit more concepts in the lecture. Although you may not have leverage on this, the scheduling of the classes was difficult in my opinion.

The fluid project was my favorite because it lets us experiment, tweak the values and see the effects of such tweaks (almost) live on the simulation. I think that encouraging students to tweak some aspects of the labs would be interesting, as it gives us a precise and deeper understanding of exactly what is going on.

Thank you for this class!